

Intérprete de funciones de lista.

Gianfranco Morán.

M-7375/1.

Estructuras de Datos y Algoritmos I.

**Facultad de Ciencias Exactas
Ingeniería y Agrimensura.**

15/9/2025.

1 - Introducción:

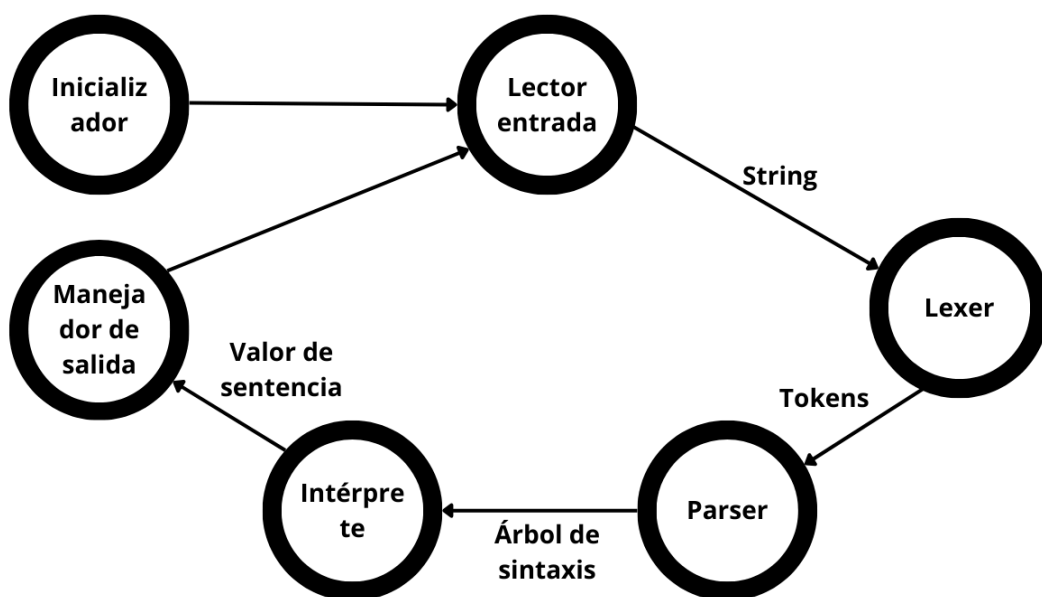
En este proyecto nos proponemos realizar un intérprete de funciones de lista. Las funciones de lista son un modelo de cálculo Turing completo, esto es, su poder de cálculo es semejante a las Máquinas de Turing. Ahora bien, nuestro objetivo es hacer un intérprete en el que se puedan definir funciones de listas, definir listas, aplicar alguna función de lista a una lista y realizar una búsqueda de funciones de lista que lleven de una lista de origen a otra de destino.

El marco teórico y la motivación del proyecto se encuentran detallados en el archivo denominado “Enunciado”.

El objetivo de este informe es hablar acerca del diseño del programa y de su implementación, dando una razón a las decisiones tomadas.

2- Diseño del proyecto:

El proyecto se encuentra dividido en 6 módulos, un inicializador, un lector de entrada, un analizador léxico o lexer, un analizador sintáctico o parser, un intérprete y un manejador de salida. Un diagrama de esto es:



Ahora bien en la parte de inicialización se crean las distintas estructuras de datos en las cuales se guardarán los datos que el usuario vaya definiendo mediante las sentencias, por ejemplo, una tabla hash con listas. Además se colocan los datos iniciales en las estructuras correspondientes.

El lector de entrada no tiene otra función más que leer desde la entrada estándar y devolver la cadena leída.

El lexer, es el módulo que recibe el string “crudo” y luego lo devuelve en una secuencia de tokens. Sus funciones son, detectar las palabras clave (deff, defl, search, apply), los símbolos que son correctos, por ejemplo el punto y coma, los corchetes. Además detectar números e identificadores (estos últimos contienen caracteres alfanuméricos). Si el string contiene alguna subcadena que tiene un símbolo que no pertenece al alfabeto, como ser una arroba, el lexer rechaza el string y no crea los tokens. Si el string cumple lo pedido, el lexer me devuelve una cola de tokens.

Ahora bien, el parser recibe esa cola de tokens que me devuelve el lexer y realiza un análisis sintáctico. Por ejemplo, el lexer me puede decir que una sentencia “Lista = defl [1 , 2] ;” es una sentencia válida, ya que no contiene ningún símbolo no permitido, sin embargo, esta sentencia no está bien armada. El parser se encarga de analizar si la sentencia está bien armada según nuestras reglas. Con el ejemplo anterior, el parser verá un token del tipo identificador como primer token, y sabemos que las sentencias empiezan con una palabra clave, entonces rechazará la sentencia. Un ejemplo de sentencia válida es “defl Lista = [1 , 2] ;”. El parser me devuelve un árbol sintáctico (AST), en el que se deja de lado la información irrelevante como ser el igual, los corchetes o el punto y coma.

El módulo intérprete es en el que se hallan los “algoritmos” de ejecución de aquellas sentencias. Este recibe un AST y realiza lo que indique la sentencia, en caso de que esta sea correcta. Por ejemplo una sentencia “deff f = Mi Oi ;” para el parser es una sentencia válida, ahora bien, si la función “Mi” no está definida, el intérprete rechaza la sentencia. En el caso de que sí esté definida, el intérprete crea la estructura de la función f y la devuelve. Como cada sentencia tendrá un retorno distinto, se crea una estructura “Valor Sentencia” y se devuelve.

Finalmente el manejador de salida recibe esta estructura Valor Sentencia y realiza las acciones según la sentencia, por ejemplo una sentencia deff correcta solo carga la función a las estructuras de datos. Pero una sentencia apply imprime la lista resultante.

Dentro de la carpeta “src” se puede ver esta división de módulos. También se tiene una carpeta en las cuales se guardan las estructuras de datos utilizadas y otra en la cual se guardan las utilidades.

3- Implementaciones:

Explicaremos los porqué de algunas decisiones tomadas en la implementación, seguiremos el mismo flujo que tiene el programa para esto.

3.1- Inicializador:

En el inicializador se utilizan tablas hash tanto para guardar funciones como listas. Esta elección de estructura se debe a que a priori no se sabe cuantas funciones o listas definirá el usuario. Además la búsqueda de estas funciones y listas es recurrente en el programa.

Cada vez que se define una función con deff, debemos verificar que las funciones que se componen se encuentren definidas, por ende debemos buscar en nuestra estructura. Cuando hacemos apply también debemos buscar tanto la función dada, como la lista en caso de que esta sea dada por id. Además en el search las listas pueden ser indicadas por id. Como vemos, nuestro programa tendrá que hacer muchas búsquedas y qué mejor que las tablas hash para tal tarea.

Ahora bien, en el inicializador también se crea una lista doblemente enlazada con funciones. Esta lista tiene el objetivo único de ser recorrida por el algoritmo que usa el search, ya que sabemos que la tabla hash no se debe recorrer.

3.2- Lexer:

En el lexer se utiliza una estructura denominada “token”, para luego dar una cola de estos, que esta es la representación de la sentencia ingresada.

La estructura del token tiene el objetivo de llevar el string a una forma más simple que aún contenga la información necesaria para validar la sentencia o en su defecto rechazarla. Por ejemplo, un string con un igual (“=”) será representado por un token cuyo valor “tipo de token” sea un entero predefinido “TOKEN IGUAL”.

Luego la elección de que el lexer me devuelva una cola de tokens, se toma sabiendo que la primera palabra que tenga mi sentencia es la primera que se “tokeniza” y es a su vez la primera que el parser debe leer para, en este caso, saber de qué sentencia se trata, desencolando esta misma. Con las demás palabras de la sentencia sucede lo mismo. Así una estructura de cola resulta perfecta para este comportamiento “el primero que entra es el primero que sale”.

Ahora bien, ¿por qué no trabajar en el parser con el string “crudo”? No trabajamos con strings en el parser ya que agrandaría aún más la tarea del parser. En ese caso, el parser además de analizar la estructura de la sentencia, debería descartar espacios, rechazar símbolos inválidos y demás. Por esta razón, delegamos esta tarea de “descarte” al lexer, y mientras leemos la cadena, tokenizamos las palabras para llevarlas a esta forma más simple.

3.3- Parser:

Hablemos ahora del parser. Este recibe la cola de tokens dada por el lexer y devuelve un árbol sintáctico (AST).

El AST tiene varios campos en los cuales guardaremos toda la información necesaria para ejecutar una determinada sentencia, además de un entero indicando el tipo de sentencia.

Las sentencias deff se representan con una estructura que tiene un string para guardar el id que se le quiere colocar y una lista que tiene las “funciones” que se

pretenden componer. Estas funciones son, o bien un string que indica el identificador de la función, o una lista de la misma estructura función (o sea que la estructura es recursiva) en la cual se guardan las funciones que se hallan dentro de una repetición. Esta representación resulta conveniente para que el parser de la sentencia deff trabaje de manera recursiva. Que a su vez también trabajará de manera recursiva el intérprete del deff. Se elige la lista como estructura ya que no sabemos cuantas funciones habrá en la composición.

Las sentencias deff son listas doblemente enlazadas en las que se guarda un id. Nuevamente se elige la lista como estructura ya que no sabemos cuántos valores tendrá la lista. Además la decisión de hacerlas doblemente enlazadas parte de las formas que tienen las primitivas, que trabajan sobre el primer valor y sobre el último.

Las estructuras de sintaxis para las sentencias apply guardan un string con la función a aplicar y el id de la lista, o en su defecto, la lista por extensión, dándole al usuario las dos posibilidades para ejecutar una sentencia apply.

Por último, las sentencias search se pueden expresar con listas por extensión, en este caso, la estructura de las sentencias search guardará dos listas de listas de enteros, una con las listas de origen y la otra con las listas de destino. Si son listas dadas por id, la estructura guarda dos listas con strings. Cabe aclarar que no se puede mezclar listar por id y listas por extensión en una sentencia search.

Estas estructuras tienen la información mínima e indispensable para ejecutar una sentencia. No como la cola de tokens, que contenía información necesaria para hacer una sentencia válida, pero innecesaria para ejecutar la sentencia al fin y al cabo, por ejemplo, el igual para definir las listas o funciones.

Entonces mientras el parser va leyendo la cola de tokens, va creando el AST para enviárselo al intérprete.

3.4- Intérprete:

El intérprete recibe el AST y devuelve una estructura “Valor Sentencia” que es la que tiene la información que se debe guardar en las tablas hash o imprimir por pantalla.

Las estructuras valor sentencia tienen 4 campos. Una estructura “Función” resultado de una sentencia deff. Una lista resultado de una sentencia deffl. Otra lista resultado de aplicar una función a una lista mediante una sentencia apply. Por último otra “Función” resultado de una sentencia search. Estas estructuras se irán completando mientras se vaya leyendo el AST y terminando de verificar que la sentencia sea correcta.

Explicaremos ahora las estructuras Función, que cabe aclarar, no son las mismas que el parser utiliza, sino que son las que se guardan “definitivamente” en la tabla hash. La estructura tiene un campo para guardar la primitiva correspondiente en caso de que sea una. O en su defecto, tiene un arreglo para guardar otra estructura llamada NodoFuncion, que no es otra cosa que las funciones que se van componiendo con el deff, por eso es que esta estructura guarda una estructura Funcion, y un entero para saber si esta es una repetición. De esta forma nos quedan unas estructuras

recursivas “circulares”. Esta recursividad resulta conveniente para definir el algoritmo del apply. Que se resuelve en la función del intérprete *apply_algoritmo*.

3.4.1- Algoritmo apply:

El algoritmo apply está completamente relacionado a la definición de las funciones dentro del programa. Si la función dada es una primitiva solo aplico la función. Ahora bien si es una composición voy a tener un arreglo de NodoFuncion, que estas a su vez tienen funciones dentro, dando lugar a llamar recursivamente a la función que aplica el algoritmo apply en cada NodoFuncion del arreglo. En caso de que la estructura NodoFuncion sea una repetición, se ejecuta el algoritmo de la repetición con la función indicada en el nodo. Este algoritmo tiene un límite de llamadas recursivas, ya que puede iterar infinitamente. En caso de que la lista dada no corresponda con el dominio de la función no se realiza el algoritmo apply.

3.4.2- Algoritmo search:

Las sentencias search se resuelven en la función del intérprete *search_algoritmo*. Este algoritmo no es otra cosa más que un BFS tomando como nodos las funciones definidas. En el algoritmo se van encolando funciones composición entre las funciones definidas y la función actual visitada por el BFS. Además este algoritmo se limita por profundidad, ya que hay casos en los que no se encontrará una función para los pares de listas dados.

Para el primer par de listas, se busca una función que cumpla lo pedido. Si esta se encuentra, se prueba esa función en el siguiente par de listas. Si la función no cumple lo pedido para cualquier par de listas siguientes se vuelve al par de listas iniciales y se sigue buscando otra función que cumpla la condición del primer par de listas. Este proceso se repite hasta encontrar la función o hasta que el límite de profundidad se alcance.

3.5- Manejador de salida:

Una vez que se tiene cargada la estructura Valor Sentencia, esta se envía al módulo manejador de salida, se actualizan las tablas hash, o en su defecto se imprime por pantalla el resultado de la sentencia.

3.6- Estructuras de datos:

En el programa se ven dos tipos de estructuras de datos, las específicas y las genéricas.

Las específicas son las estructuras de datos que se crean con un fin específico, para un cierto módulo del programa, por ejemplo el AST para el parser y el intérprete, la estructura Valor Sentencia para el intérprete y el manejador de salida, y demás ejemplos. Estas estructuras tienen interfaces básicas de creación, destrucción y, en

algunos casos, copia. Esta interfaz básica es de esta forma debido a que no se busca que la estructura sea únicamente manipulada desde su interfaz a diferencia de las estructuras genéricas, sino que se busca que sea manipulada desde el módulo del programa dueño de esta. Por ejemplo, los tokens son creados y cargados por el lexer, para luego ser leídos por el parser. Una manipulación únicamente desde la interfaz no tendría tanto sentido al tratarse de estructuras que se crean específicamente para un módulo y complicaría un poco más el programa.

Las estructuras genéricas en este programa son listas doblemente enlazadas, tablas hash y colas. Todas estas son estructuras de un propósito más bien general y se usan en distintos módulos del programa, por eso se hace que se deba respetar la interfaz de la misma “opacando” la estructura.

La estructura de lista dentro del programa se usa como una lista propiamente dicha para guardar, recorrer y trabajar en general, con distintos datos dentro del programa, como por ejemplo, definir las colas. Pero también la usamos para definir las listas dentro del intérprete, por eso es que tienen algunos campos “raros” como ser el id, que no se usa en otro caso que no sea el de guardar una lista definida por el usuario.

3.7- Otros:

Un punto que puede resultar raro, es el hecho de que las funciones destrucción devuelvan un puntero a la misma estructura que están destruyendo. Esto se hace con el único fin que todo “tipe” para hacer NULL a la estructura que se está destruyendo. En algunos casos este comportamiento no se usa y simplemente se ignora el retorno, pero en otros casos hacer NULL a la estructura liberada resulta útil, ya que en todo el programa las funciones se “protegen” de los punteros NULL.

4- Tests:

Los casos de prueba que se realizaron en el programa fueron las sentencias que se hallan en la carpeta test bajo el nombre de “test.txt”.

5- Conclusión:

En resumen, el programa implementa un intérprete de funciones de lista que permite definir funciones de lista, listas, aplicar funciones a listas y buscar funciones entre pares de listas dados. Todo esto utilizando estructuras de datos aprendidas en la materia, además de nuevas estructuras “inventadas” con el objetivo de resolver los distintos problemas del intérprete. Estas últimas no hubiesen sido posible de concebir si no se conocieran las estructuras de datos estudiadas en clase. Además de las estructuras de datos, se aplicaron conocimientos de algoritmia, como ser el BFS para el search.